

Making dimascore.com Feel Instant: Tactical Next.js 16 Patterns from Sofascore, FotMob & OneFootball

TL;DR

- **Do NOT migrate to Pages Router.** The "click and content is already there" feel of Sofascore/FotMob/OneFootball comes almost entirely from a **client-side data cache (React Query/SWR) that prefetches likely-next data and seeds the detail cache from the list cache** — not from the router choice. Adopt that pattern on data-heavy pages while staying on App Router.
- **Fix the actual root cause first:** your competition page is force-dynamic because of `searchParams` access, which disables edge caching and pays ~1.7s of sequential DB queries on every navigation. Isolate `searchParams` into a small Client Component under `<Suspense>`, parallelize your `timedQuery` calls with `Promise.all`, and enable `cacheComponents` (PPR) so a static shell streams instantly. This is the highest-impact, lowest-effort change.
- **Pursue a hybrid end state:** App Router + `cacheComponents`/PPR for the static shell and SEO, plus a thin Client Component + React Query layer (fed by Route Handlers returning small JSON) for the live, navigable, "0-network-request-onward" tabs. This matches what the benchmark sites actually do.

Key Findings

What the benchmark sites actually do (confirmed)

- **All three serve data over their own internal HTTP JSON/RPC APIs, fetched client-side after first paint.** FotMob uses a REST API at `www.fotmob.com/api/` (e.g. `matchDetails?matchId=`, `leagues?id=&season=`, `matches?date=YYYYMMDD`), now gated behind a signed `x-mas`/`x-fm-req` header (returns `401` without it). Sofascore uses a REST API at `api.sofascore.com/api/v1/...` (e.g. `/sport/football/events/live`), now behind Cloudflare bot protection that 403s plain REST calls. OneFootball uses "XPA," a **Protobuf-based RPC API** — its public licenses page ships the dependency `@motain/xpa-proto-files-web` ("motain" is OneFootball GmbH's legacy org) — not plain REST or GraphQL. (GitHub)
- **OneFootball is confirmed to be built on Next.js** (its licenses page lists the full `@next/*` toolchain including `@next/bundle-analyzer`, `@next/env`, `@next/font`). App-vs-Pages Router could not be confirmed for any of the three from authoritative sources; the working assumption (Pages Router, based on `__NEXT_DATA__`) remains unverified.
- **All three use dedicated image CDN hosts separate from the app origin:** `images.fotmob.com` (confirmed pattern `image_resources/logo/teamlogo/{id}_xsmall.png`), `img.sofascore.com` (e.g.

`api/v1/team/{id}/image`), and `image-service.onefootball.com`. None rely on Next.js image optimization.

- **Live-update transport (WebSocket vs SSE vs polling) could not be definitively confirmed for any of the three.** Third-party scrapers poll Sofascore's `events/live` endpoint at self-imposed 25–30s intervals to avoid Cloudflare bans, but this reflects scraper etiquette, not necessarily the sites' own refresh cadence. Treat "they poll every N seconds" as unconfirmed.

Why your competition page is slow (the real diagnosis)

Your ~1.7s render that is "never edge-cached (forced dynamic by searchParams access)" is a textbook Next.js anti-pattern. Accessing `searchParams` in a Server Component page opts the **entire route** into dynamic rendering. Combined with sequential `await timedQuery(...)` calls, you pay full DB latency on every single navigation, server-side, with no shell shown until everything resolves. The benchmark sites never do server work on tab-to-tab navigation at all — the data is already in a client cache.

Details

1. Data fetching patterns — adopt React Query (TanStack Query)

The single most impactful adoptable pattern is a **client-side query cache that (a) dedupes, (b) serves stale-while-revalidate, and (c) is pre-seeded so onward navigations cost zero network requests**. TanStack Query is the right choice over SWR here because you need fine-grained `staleTime`/`gcTime` control, query invalidation, and devtools for a data-heavy app; SWR is lighter (and refreshes via `refreshInterval`) but you'll outgrow its controls. Both implement stale-while-revalidate; React Query adds the cache-manipulation primitives the seeding/prefetch tricks below depend on.

Cache key structure — co-locate keys in a single module to avoid typos and enable targeted invalidation:

```
ts
// lib/queryKeys.ts
export const qk = {
  match: (id: string) => ['match', id] as const,
  matchStats: (id: string) => ['match', id, 'stats'] as const,
  competition: (id: string, season: string) => ['competition', id, season] as const,
  standings: (id: string, season: string) => ['competition', id, season, 'standings'] as const,
  team: (id: string) => ['team', id] as const,
};
```

Seed the detail cache from the list cache (the "0 network requests" trick). When you render a competition's fixture list, you already hold each match's summary. Push it into the match cache so clicking a fixture renders instantly:

ts

```
// after fetching the fixtures list:
fixtures.forEach(fx => {
  queryClient.setQueryData(qk.match(fx.id), seedFromFixture(fx));
});
```

The match page's `useQuery(qk.match(id))` then finds data immediately and only revalidates in the background. This is TkDodo's documented "seeding the query cache" pattern and is exactly how list→detail feels instant on the benchmark sites. [TkDodo](#)

Prefetch likely-next data while rendering the current page using

`queryClient.prefetchQuery` (or `ensureQueryData`, which ignores `staleTime` and returns cached data if present). Trigger it on render for high-probability targets (e.g. the top fixtures on a competition page) and on hover/`onMouseDown` for the rest:

tsx

```
<Link href={` /match/${m.id}`}
  onMouseEnter={() => queryClient.prefetchQuery({
    queryKey: qk.match(m.id), queryFn: () => fetchMatch(m.id), staleTime: 10_000
  })}>
```

2. Pages Router vs App Router — stay on App Router

The community evidence is real that **default App Router navigation can feel slower than Pages Router** (multiple Vercel GitHub discussions report ~2× slower TTFB in self-hosted SSR; the core reason is that App Router streams RSC payloads from the server on each navigation, whereas Pages Router ships the next page's JS upfront). **But the fix is not migrating** — it's adopting client-side data caching on top of App Router, which neutralizes the difference. Migrating ~100 pages across 3 locales to Pages Router would be a large rewrite that abandons RSC's SEO and bundle-size benefits, and you'd still have to build the same client cache layer to get the "instant" feel. Note also: navigating between App Router and Pages Router pages causes a **hard navigation** (prefetch doesn't cross routers), so a partial migration is the worst of both worlds.

3. Image strategy — move off Next.js image optimization

All five tested sites serve images from dedicated CDN hosts, and you should too. On Vercel, the built-in `next/image` optimizer is metered and gets expensive at scale. In HowdyGo's documented case ("Cutting Vercel Costs by 80% without compromising UX or DX"), self-hosting an AWS Lambda + S3 + CloudFront optimizer "reduced our overall hosting costs by over 80%," staying within the AWS Free Tier for ~28,000 images/month; they calculated it was "25 times cheaper than Cloudflare and 250 times cheaper than Vercel" at that volume (~\$0.02 per 1,000 images after the free tier). For football logos/headshots — a **finite, slow-changing set** — the best fit is: [HowdyGo](#) [HowdyGo](#)

- **Cheapest/simplest:** upload your API-Football images to an object store and front it with **Bunny CDN** (Standard Network from \$0.01/GB in Europe/North America; the Bunny Optimizer resize + WebP/AVIF add-on is a flat \$9.50/month per site for unlimited optimizations) or **Cloudflare**. Set `images.unoptimized` or use a custom Next.js loader pointing at the CDN host. `CostBench`
- Because the image set is small and static, you can pre-process once rather than transform per-request — avoiding per-optimization fees entirely.

4. API/data layer — small per-entity JSON via Route Handlers (BFF)

The benchmark sites use **per-entity REST/RPC endpoints returning small JSON/Protobuf**, fetched on demand. Replicate this with App Router **Route Handlers** (`app/api/.../route.ts`) acting as a Backend-for-Frontend over your Neon/Drizzle layer. Each handler returns exactly what one tab needs (match header, standings table, fixture rows) — keep payloads lean so they cache well at the edge and hydrate React Query fast. Parallelize independent reads with `Promise.all` inside each handler. Avoid Server Actions for reads — Next.js docs explicitly warn they are queued and run sequentially, making them a poor fit for data fetching. `Next.js`

5. Live updates — keep Pusher, scope it tightly

You already have Pusher (WebSocket). That is the correct transport for live scores and is better than re-fetching pages. The key tactical pattern: **on a Pusher event, call `queryClient.setQueryData` to patch just the changed match/score in the cache** — never invalidate-and-refetch the whole page. For non-live data (standings, lineups), lean on React Query's `staleTime` + (optionally) `refreshInterval`. If you ever want to shed Pusher cost for one-way score pushes, SSE is the lighter-weight alternative (plain HTTP, auto-reconnect, passes through CDNs without the WebSocket upgrade handshake), but there's no need to change what works.

6. Next.js 16 App Router patterns for SPA-like behavior

- `cacheComponents: true` (**PPR, stabilized in Next.js 16, shipped Oct 2025 with React 19.2 and detailed at Next.js Conf on Oct 22, 2025; the old `experimental.ppr` flag was removed in favor of the top-level `cacheComponents`**). This is the headline feature for your problem: it renders a static shell instantly and streams only the dynamic, `<Suspense>`-wrapped parts. Per the Next.js docs, with `cacheComponents` enabled "Next.js uses React's `<Activity>` component to preserve component state during client-side navigation. Rather than unmounting the previous route when you navigate away, Next.js sets the Activity mode to 'hidden'." Caching is opt-in via `'use cache'` + `cacheLife`. `Next.js`
- **Incremental prefetching + layout deduplication (new default in 16).** A page with many fixture links now downloads the shared layout once, not once per link, and cancels prefetches when links scroll out of view. This directly helps your competition/fixture lists. `Next.js`

- **<Link prefetch>**: default ("auto") prefetches static routes fully and dynamic routes down to the nearest `loading.tsx`. For your dynamic pages, **add a `loading.tsx` at each segment** so prefetch has something to cache and navigation shows an instant skeleton.
- **`use()` hook + promise passing**: fetch in a Server Component without awaiting, pass the promise to a Client Component that unwraps it with `use()` under `<Suspense>` — streams data without blocking the shell.
- **View Transitions (experimental in 16, React 19.2)**: `experimental.viewTransition` + `<ViewTransition>` gives native-feeling shared-element morphs (e.g. team crest → team page header) for ~2 CSS declarations. Nice polish, not a perf fix. `Next.js`

7. Concrete reference implementations

- **NextFaster** (`github.com/ethanniser/NextFaster`) is the canonical "instant" App Router demo. Its custom `<Link>` navigates on `onMouseDown` rather than click, and prefetches the destination's **images** on hover via a dedicated `prefetch-images` API route. The `onMouseDown` trick is grounded in measurement: Vercel Next.js issue #39934 notes API-dependent pages take "~170ms" before render and that prefetching on `onmousedown` makes "page transitions >2X faster." Both tricks are directly adoptable. `Ravi Coding`
- **TkDodo's "Seeding the Query Cache"** documents the list→detail cache-seeding pattern.
- **TanStack Query's official Next.js App Router examples** (prefetch + `HydrationBoundary`/`dehydrate`) show the server-prefetch-then-hydrate pattern for first paint. `TanStack`

8. Anti-patterns to avoid

- **Accessing `searchParams`/`cookies()`/`headers()` high in a Server Component** — opts the whole route into dynamic rendering. This is your current bug. Push it into a child Client Component under `<Suspense>` (or unwrap with `use()`), or use `connection()` only where you genuinely need request-time rendering. `Next.js`
- **Sequential `await` data fetching** (request waterfalls) — parallelize with `Promise.all`.
- **An empty `<Suspense>` fallback above `<body>` in the root layout** — Next.js docs warn this defers the entire app to request time, killing the static shell.
- **Using Server Actions for data reads** — they're queued/sequential.
- **Over-aggressive `prefetch={true}` on long link lists** — burns Vercel Fast Origin Transfer. Mike Bifulco documented "more than sixty links... prefetching their target pages the moment someone loaded the homepage. On every visit," noting Next.js 16 "made prefetching behavior more aggressive by default." Use hover/viewport defaults or a hover-only wrapper. `Mike Bifulco`

- `onSuccess`/`onError` callbacks on `useQuery` — removed in v5 as bug-prone; do side effects elsewhere. `TkDodo`

Recommendations (ranked by impact ÷ effort)

Stage 1 — Fix the dynamic-rendering bug (days, highest impact).

1. Move `searchParams` access out of the competition page Server Component into a small `'use client'` child wrapped in `<Suspense>` (or read it via `use()` in a client child). This alone can restore edge-cacheability of the shell.
2. Replace sequential `await timedQuery(...)` with `Promise.all` for independent queries.
3. Add `loading.tsx` at each dynamic segment (`/[locale]/competition/[id]`, `/match/[id]`, etc.) so prefetch + navigation show an instant skeleton. *Benchmark to hit: shell TTFB < 200ms from edge; competition navigation shows skeleton in < 100ms.*

Stage 2 — Add the client cache + prefetch layer (1-2 weeks, delivers the "instant" feel). 4. Introduce TanStack Query with a co-located query-key module and a Route-Handler BFF returning small per-tab JSON. 5. Seed the match/team detail cache from list data (`setQueryData`) and `prefetchQuery` top targets on render + hover. 6. Wire Pusher events to `setQueryData` patches (no full refetch). *Benchmark: Match→Match and tab switches cost 0 network requests when seeded; live score updates patch in < 1s without layout reflow.*

Stage 3 — Enable `cacheComponents`/PPR (1-2 weeks, structural). 7. Set `cacheComponents: true`, wrap dynamic regions in `<Suspense>`, mark slow-changing data (`'use cache'` + `cacheLife`) — standings, fixtures, team metadata are ideal cache candidates since your cron-ingested data changes on a known cadence. 8. Verify each route shows the  (partial prerender) marker in build output. *Benchmark: competition/team/player pages prerender a static shell; only live regions stream.*

Stage 4 — Polish (ongoing). 9. Move images to Bunny/Cloudflare CDN with a custom loader; pre-process the finite logo/headshot set. 10. Adopt NextFaster's `onMouseDown` navigation + image-prefetch-on-hover. 11. Optionally add View Transitions for shared-element morphs.

What would change these recommendations: If your competition pages don't actually need `searchParams` for the cached view (e.g. tab state could live in the client), skip straight to making them fully static + client-tabbed. If Neon query latency dominates even when parallelized (> ~500ms), the bottleneck is the DB, and you should pre-compute/cache aggregates (standings, top scorers) in a `'use cache'` layer or a materialized table rather than chasing front-end tricks.

Caveats

- **The three benchmark sites' live-update transport is unconfirmed.** I could not verify WebSocket vs SSE vs polling for any of them; do not copy a specific polling interval on

the assumption that "that's what Sofascore does." The 25–30s figure is scraper etiquette against Cloudflare, not a confirmed front-end cadence.

- **App-vs-Pages Router is confirmed only for OneFootball (Next.js, router unknown).** The premise that all three are Pages Router is plausible but unverified; it does not change the recommendation, since the "instant" feel derives from the client cache, not the router.
- **App Router "slower than Pages Router" reports are mostly from self-hosted/standalone deployments;** on Vercel the gap largely disappears, and PPR + prefetching close it further. Your Vercel hosting is favorable here.
- **PPR/`cacheComponents` is young (stable as of Next.js 16, Oct 2025).** Budget time for the `<Suspense>`-boundary migration and the "Uncached data was accessed outside of `<Suspense>`" build errors it surfaces.
- **Image-CDN cost figures are from third-party blog case studies** (e.g. HowdyGo at ~28,000 images/month) and vendor pricing pages; model your own API-Football image volume before committing.
- **FotMob and Sofascore actively block unofficial API access** (signed headers, Cloudflare). Cited endpoint patterns are for architectural reference only — your data source is API-Football, so this is about *shape and delivery*, not reusing their endpoints.